

Section 76 — Suffix Arrays And LCP Structure

1. Why sort suffixes?

A substring is a prefix of some suffix. After suffixes are sorted, all suffixes beginning with the same pattern form one contiguous interval. This single geometric fact turns substring questions into interval questions.

This lesson develops Problems A–C. Problem D is deliberately reserved for independent discovery and the editorial.

2. Problem A: build the order

A direct comparison of suffixes repeats work. Prefix doubling instead stores a rank for every block of length 2^h . To rank length $2^{(h+1)}$, sort the pair

```
(rank[i], rank[i + 2h]).
```

The pair is a compressed model of the whole longer block: everything relevant to lexicographic comparison crosses the round boundary through two integer ranks.

After the suffix array, Kasai's algorithm computes adjacent LCP values. When moving from suffix i to suffix $i+1$, an already-known common prefix can shrink by at most one. Reusing that prefix makes the total number of successful character comparisons linear.

Invariant. At the start of doubling round h , equal ranks mean equal length- 2^h prefixes, and rank order equals their lexicographic order.

Complexity here is $O(n \log^2 n)$ with comparison sorting and $O(n)$ for Kasai. Counting/radix sorting reduces construction to $O(n \log n)$.

A doubling trace on banana

At length one, positions have ranks 1,0,2,0,2,0 for letters b,a,n,a,n,a. In the length-two round, position 1 is represented by pair (0,2), as is position 3: both prefixes are **an**. Position 5 uses (0,-1), so suffix **a** comes first. Each round doubles the text certified by two ranks; once all ranks differ, the order is final.

The sentinel rank -1 must be smaller than every real rank. Otherwise a longer suffix can be placed before an equal prefix that ends earlier.

Why Kasai is linear

If suffix i has adjacent LCP h , removing its first character leaves a common prefix of at least $h-1$ for suffix $i+1$ and some nearby suffix. Kasai starts there. Its reused length increases only after a successful comparison and decreases by at most one per text position, so total comparisons are linear by a potential argument.

3. Problem B: repeated at least k times

If a substring occurs in k suffixes, those suffixes occupy a contiguous suffix-array block. Their common prefix length is the minimum of the $k-1$ adjacent LCP values inside the block. Therefore:

1. slide a window of $k-1$ over the LCP array;
2. maintain its minimum with a monotone deque;
3. maximize that minimum.

The hard step is recognizing that a global collection of occurrences becomes a local contiguous block after sorting.

If one suffix-array block has adjacent LCP values 5,3,4, all four suffixes share exactly $\min(5,3,4)=3$ characters. For $k=3$, consecutive windows use [5,3] and [3,4]; a monotone deque maintains these minima as one value enters and one leaves.

4. Problem C: pattern occurrence queries

For a pattern p , suffixes smaller than p come first, suffixes beginning with p form the answer interval, and greater suffixes follow. Two binary searches find the interval boundaries.

Do not compare p with an entire copied suffix. Compare characters in the original text, and treat “the pattern ended” as equality even if the suffix continues.

With ordinary comparisons, a query costs $O(|p| \log n)$. More advanced LCP-aware searches can reduce repeated comparisons.

When pattern **ana** is compared with suffix **anana**, it matches for occurrence search because the pattern ended. When suffix **an** is compared with **ana**, the suffix is smaller because it ended first. Encode these cases explicitly rather than relying on ordinary whole-string comparison.

Implementation notes

Never construct suffix strings. C++ can sort positions by rank pairs; Python can sort tuples, though counting/radix sorting may be needed under tighter limits. Build the inverse suffix array once for Kasai.

State the LCP convention. Here $\text{lcp}[i]=\text{LCP}(\text{sa}[i], \text{sa}[i+1])$, so its length is $n-1$. Templates using a shifted length- n array require different window and RMQ indices.

Proof habits and wrong approaches

- Prove what one rank means before the next doubling round.
- Prove that all suffixes prefixed by one pattern are contiguous.
- A prefix shared by many suffixes is the **minimum** adjacent LCP in their block, not the maximum.
- Hashing compares chosen substrings but does not organize all suffixes.
- Binary search is valid only after proving a monotone feasibility predicate.

Stress-testing workflow

For strings of length at most nine, explicitly sort every suffix, compute LCP character by character, and enumerate every substring occurrence. Include unary, periodic, and all-distinct strings, patterns longer than the text, and the empty LCP array of a one-character text.

5. A reusable discovery checklist

- Is every candidate a prefix of some suffix?
- Does sorting make all witnesses contiguous?
- Is the answer over a suffix-array interval a minimum of adjacent LCPs?
- Are many neighboring interval minima required? Try a deque, RMQ, or offline DSU.
- Is feasibility monotone in a candidate length? Try binary search.

Exercises

- A — construct suffix order and LCP.
- B — turn repeated occurrences into a sliding LCP minimum.
- C — turn a pattern into a suffix-array interval.
- D — new transfer problem; no lesson derivation is given.